

markpublish

Benutzerhandbuch

Moderne PDF- und HTML-Dokumentenerstellung aus Markdown

Dieses Handbuch bietet eine praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, Template-Anpassung und Dokumentengenerierung mit markpublish.

AUTOR
Frank Winter

STATUS
Freigegeben

VERSION
1.0.0

DATUM
31.08.2026

COPYRIGHT
© 2026 Frank Winter

Inhaltsverzeichnis

1 Einführung & Architektur	5
1.1 Motivation & Zielsetzung	5
1.1.1 Kernvorteile auf einen Blick:	5
1.2 Die Verarbeitungs-Pipeline	5
2 Installation & Schnellstart	7
2.1 Voraussetzungen	7
2.2 Installation via pip	7
2.3 Erstes Projekt initialisieren	7
2.4 Dokument erstellen	7
3 Die CLI-Befehle	10
3.1 Befehlsübersicht	10
3.2 markpublish build	10
3.2.1 Argumente & Optionen:	10
3.2.2 Beispiele:	10
3.3 markpublish init	11
3.4 markpublish templates	11
3.5 markpublish export-template	11
3.6 markpublish labels	11
3.6.1 Argumente & Optionen	11
4 Konfigurations-Leitfaden	14
4.1 Das document-Objekt	14
4.1.1 Automatische Datumsauflösung	14
4.1.2 Zusätzliche Metadatenfelder	14
4.2 Kapitel, Unterkapitel und Parts	14
4.2.1 Kapitel-Definition	14
4.2.2 Hierarchische Unterkapitel	15
4.2.3 Übergeordnete Abschnitte (Parts / Blöcke)	15
5 Das Template- und Design-System	17
5.1 Theme-basierte Verzeichnisstruktur	17
5.2 Die 3-stufige Auflösungs-Hierarchie	17
5.2.1 Konfiguration des gemeinsamen Template-Ordners	17
5.3 Statische Texte und Sprache	18
5.3.1 Die 4-stufige i18n-Kaskade	18
5.3.2 Das gemeinsame Format	19
5.3.3 Beispiel: PDF und HTML unterschiedlich beschriften	19
5.3.4 Ebene 4: Texte für ein einzelnes Dokument	20
5.3.5 Die aufgelöste Tabelle ansehen	20
5.3.6 Verfügbare Schlüssel	21
5.4 2-zeilige Kopf- und Fußzeilen	22

6 Markdown-Features & Syntax	24
6.1 Code-Blöcke & Syntax Highlighting	24
6.2 Tabellen	24
6.3 GitHub-Style Callout-Boxen (Alerts)	24
6.4 Tasklisten	25
 Anhänge	
6.5 Anhang A: YAML-Schema-Referenz	27
6.5.1 Dokumenten-Eigenschaften (document)	27
6.5.2 Kapitel- und Part-Eigenschaften (chapters)	28
6.6 Anhang B: Fehlerbehebung & FAQ	28
6.6.1 WeasyPrint & GTK-Laufzeitumgebung auf Windows	28
6.6.1.1 Problem: cannot load library 'libgobject-2.0-0'	28
6.6.2 Bilder werden im PDF nicht angezeigt	28
6.6.2.1 Problem: Leere Bildrahmen oder fehlende Grafiken	28
6.6.3 Inhaltsverzeichnis zeigt falsche Seitenzahlen	29

KAPITEL 1

Einführung & Architektur

Überblick über Zielsetzung, Kernkonzepte und die modulare Verarbeitungs-Pipeline.

1 Einführung & Architektur

Willkommen beim **markpublish Benutzerhandbuch**. `markpublish` ist ein modernes, quelloffenes Publishing-Werkzeug, das aus einfachen Markdown-Dateien professionell gesetzte **PDF-Publikationen** und **HTML-Vorschauen** erzeugt.

1.1 Motivation & Zielsetzung

Viele Dokumentationswerkzeuge erfordern entweder komplexe LaTeX-Setups oder beschränken sich auf reine HTML-Webseiten. `markpublish` verbindet die Einfachheit von Markdown mit der typografischen Präzision von **CSS Paged Media** über die [WeasyPrint](#)-Engine.

1.1.1 Kernvorteile auf einen Blick:

- **Modulare Kapitel:** Jedes Kapitel wird als separate Markdown-Datei gepflegt.
- **Hierarchische Struktur:** Beliebig tief schachtelbare Unterkapitel und übergeordnete Abschnitte (*Parts*).
- **Deklarative Steuerung:** Ein zentrales Manifest (`markpublish.yaml`) steuert Inhalt, Metadaten und Layout.
- **W3C CSS Paged Media:** Exakte Kontrolle über `@page` -Ränder, 2-zeilige Kopf- und Fußzeilen, Seitenzahlen und Trennseiten.
- **Erweiterbare Pipeline:** Saubere Trennung zwischen Parser, Renderer und Template-Ebenen.

1.2 Die Verarbeitungs-Pipeline

Die Architektur von `markpublish` folgt einem mehrstufigen Prozess:

1. **Konfigurations-Lader** (`config.loader`): Liest die YAML-Struktur ein, validiert Datentypen und löst dynamische Variablen (z. B. `date: auto`) auf.
2. **Template-Resolver** (`templates.resolver`): Sucht nach der 3-stufigen Priorität (*User > Common/Workspace > Package*) nach dem passenden Theme für das Zielformat.
3. **Markdown- & TOC-Engine** (`markdown.engine`): Parst Markdown mit erweiterten Erweiterungen (Callouts, Tabellen, Pygments-Syntax-Highlighting), vergibt konsistente Überschriftennummern (`1.1` , `1.2`) und baut Inhaltsverzeichnisse auf.
4. **Renderer** (`renderers.pdf` & `renderers.html`): Übergibt die gerenderten HTML- und CSS-Fragmente an WeasyPrint für den PDF-Export oder speichert eigenständiges, responsives HTML.

KAPITEL 2

Installation & Schnellstart

Schritt-für-Schritt-Anleitung zur Installation und Erstellung des ersten Dokuments.

AUF EINEN BLICK

2.1 Voraussetzungen	7
2.2 Installation via pip	7
2.3 Erstes Projekt initialisieren	7
2.4 Dokument erstellen	7

2 Installation & Schnellstart

In diesem Kapitel erfahren Sie, wie Sie `markpublish` installieren und in weniger als zwei Minuten Ihr erstes Dokument kompilieren.

2.1 Voraussetzungen

`markpublish` benötigt **Python 3.10 oder neuer**. Es läuft nativ unter: - **Windows** (10, 11, Server) - **macOS** (Intel und Apple Silicon) - **Linux** (Debian, Ubuntu, Fedora, Arch, etc.)

2.2 Installation via pip

Installieren Sie das Paket über den Python-Paketmanager:

```
pip install markpublish
```

Für Entwickler oder zur Installation aus dem Quellcode:

```
git clone https://github.com/your-username/markpublish.git
cd markpublish
pip install -e ".[dev]"
```

2.3 Erstes Projekt initialisieren

Verwenden Sie den Befehl `init`, um eine neue Dokumentenstruktur zu erzeugen:

```
markpublish init mein-leitfaden --title "Mein Leitfaden"
cd mein-leitfaden
```

Dieser Befehl erstellt die folgende Verzeichnisstruktur:

```
mein-leitfaden/
├─ markpublish.yaml          # Das Dokumenten-Manifest
├─ chapters/                 # Markdown-Kapiteldateien
│   ├─ 01_introduction.md
│   ├─ 02_architecture.md
│   │   ├─ 02_1_details.md
│   │   └─ 03_appendix.md
```

2.4 Dokument erstellen

Rendern Sie Ihr Dokument mit dem `build`-Befehl:

```
# PDF erstellen (Standard)
markpublish build

# HTML-Vorschau erzeugen
markpublish build --target html

# Sowohl PDF als auch HTML in einem Durchgang bauen
markpublish build --target all
```

Die fertigen Dateien werden direkt im Projektordner abgelegt.

KAPITEL 3

Die CLI-Befehle

Detaillierte Erläuterung aller Befehle: build, init, templates und export-template.

AUF EINEN BLICK

3.1 Befehlsübersicht	10
3.2 markpublish build	10
3.3 markpublish init	11
3.4 markpublish templates	11
3.5 markpublish export-template	11
3.6 markpublish labels	11

3 Die CLI-Befehle

`markpublish` bietet eine schlanke, intuitive Befehlszeilenschnittstelle (CLI), die über `markpublish` oder die Kurzform `mpub` aufgerufen werden kann.

3.1 Befehlsübersicht

BEFEHL	KURZBESCHREIBUNG
<code>markpublish build</code>	Kompiliert das Dokument zu PDF und/oder HTML
<code>markpublish init</code>	Erzeugt ein neues Projekt mit Beispieldateien
<code>markpublish templates</code>	Listet alle gefundenen Templates und deren Quellen auf
<code>markpublish export-template</code>	Exportiert ein Template zur individuellen Anpassung
<code>markpublish labels</code>	Zeigt die aufgelösten statischen Texte und ihre Herkunft

3.2 `markpublish build`

Kompiliert eine `markpublish.yaml`-Konfiguration.

```
markpublish build [CONFIG_FILE] [OPTIONEN]
```

3.2.1 Argumente & Optionen:

- `CONFIG_FILE` (*optional*, *Standard*: `markpublish.yaml`): Pfad zur YAML-Datei.
- `--target / -t` (*Standard*: `pdf`): Zielformat (`pdf`, `html` oder `all`).
- `--output / -o`: Benutzerdefinierter Ausgabepfad (Datei oder Verzeichnis).
- `--templates-dir`: Spezifischer Pfad zu einem gemeinsamen Template-Verzeichnis.

3.2.2 Beispiele:

```
# Standard-Build aus aktuellem Verzeichnis
markpublish build

# HTML-Version in ein bestimmtes Zielverzeichnis ausgeben
markpublish build markpublish.yaml -t html -o dist/

# Benutzerdefiniertes Template-Verzeichnis nutzen
markpublish build --templates-dir /shared/company-templates
```

3.3 markpublish init

Initialisiert ein neues Projektverzeichnis.

```
markpublish init [ZIELORDNER] [--title "Titel"]
```

3.4 markpublish templates

Zeigt eine tabellarische Übersicht aller Templates auf dem System und deren Prioritätsstatus:

```
markpublish templates  
markpublish templates --target pdf
```

3.5 markpublish export-template

Kopiert das eingebaute Standard-Template in Ihr lokales Arbeitsverzeichnis:

```
markpublish export-template default templates
```

3.6 markpublish labels

Zeigt, welcher statische Text am Ende gilt und aus welcher Ebene der Label-Kaskade er stammt. Nützlich, sobald ein Theme oder ein Dokument eigene Texte mitbringt und nicht mehr offensichtlich ist, welche Ebene gewinnt.

```
markpublish labels                # alle Schlüssel, Zielformat PDF  
markpublish labels --target html  # Kaskade für die HTML-Ausgabe  
markpublish labels --overridden   # nur überschriebene Texte  
markpublish labels pfad/zu/markpublish.yaml
```

3.6.1 Argumente & Optionen

PARAMETER	TYP	STANDARD	BESCHREIBUNG
<code>config_file</code>	Pfad	<code>markpublish.yaml</code>	Pfad zur Konfigurationsdatei
<code>--target</code> , <code>-t</code>	String	<code>pdf</code>	Zielformat, dessen Kaskade gezeigt wird (<code>pdf</code> oder <code>html</code>)
<code>--templates-dir</code>	Pfad	–	Alternativer Template-Ordner
<code>--overridden</code>	Flag	<code>aus</code>	Nur Texte anzeigen, die Theme oder Dokument ändern

Die Spalte *Source* nennt die Ebene: `i18n.yaml (de)` für den Programmstandard, den Pfad einer Theme- oder Zielformat- `i18n.yaml` oder `document.i18n (de)` . Unter der Tabelle steht, in welchen Dateien nach Overrides gesucht wurde und welche existieren.

 Tipp

`--override` beantwortet die häufigste Frage direkt: *Was weicht in diesem Projekt überhaupt vom Standard ab?*

KAPITEL 4

Konfigurations-Leitfaden

Deklarative Definition von Dokumentenmetadaten, Kapiteln und Hierarchien.

AUF EINEN BLICK

4.1 Das document-Objekt	14
-------------------------------	----

4 Konfigurations-Leitfaden

Die Datei `markpublish.yaml` ist das zentrale Steuerungsdokument jeder Publikation. Sie gliedert sich in Metadaten, Template-Einstellungen und Kapiteldefinitionen.

4.1 Das `document` -Objekt

Unter `document:` werden alle globalen Metadaten und Schalter hinterlegt:

```
document:
  title: "markpublish Benutzerhandbuch"
  subtitle: "Moderne PDF- und HTML-Dokumentenerstellung"
  summary: "Kurze Zusammenfassung für Deckblatt und Metadaten."
  author: "Frank Winter"
  organisation: "WASDCAT Games"
  date: "auto"                # "auto" (Tagesdatum) oder festes Datum "2026-08-31"
  version: "1.0.0"
  language: "de"              # Sprachcode für Silbentrennung & Formatierung

# Globale Schalter
cover: true                   # Deckblatt an/aus
toc: true                     # Globales Inhaltsverzeichnis
autonum_type: "decimal"       # "decimal", "roman", "Legal", "none"
header: true                  # 2-zeilige Kopfzeile
footer: true                  # 2-zeilige Fußzeile
```

4.1.1 Automatische Datumsauflösung

Wird `date: "auto"` oder `date: "today"` angegeben, generiert `markpublish` automatisch das aktuelle Datum im passenden Sprachformat: - `language: "de"` $\rightarrow$ 31.08.2026 - `language: "en"` $\rightarrow$ 2026-08-31

4.1.2 Zusätzliche Metadatenfelder

Beliebige zusätzliche Schlüssel (z. B. `status: "Entwurf"`, `department: "IT-Architektur"`) können frei definiert werden und stehen in allen Jinja2-Templates über `document.<fieldname>` zur Verfügung.

4.2 Kapitel, Unterkapitel und Parts

`markpublish` unterstützt eine flexible und intuitive Strukturierung von Dokumenten.

4.2.1 Kapitel-Definition

Ein einfaches Kapitel wird mit `file:`, optionalem `title:` und `summary:` angegeben:

```
chapters:
- file: "chapters/01_intro.md"
  title: "Einleitung"
  summary: "Zielsetzung des Leitfadens."
  divider_page: true          # Erzeugt eine Trennseite vor dem Kapitel
  toc: false                  # Kein kapitelweises Mini-TOC
```

4.2.2 Hierarchische Unterkapitel

Um ein Unterkapitel zu definieren, rücken Sie einfach weitere Kapitel unter `chapters:` ein:

```
chapters:
- file: "chapters/02_architecture.md"
  title: "Systemarchitektur"
  divider_page: true
  toc: 2                      # Mini-TOC bis Überschriftstiefe 2
  chapters:
  - file: "chapters/02_1_backend.md"
    title: "Backend Services"
    divider_page: false
  - file: "chapters/02_2_frontend.md"
    title: "Frontend Client"
    divider_page: false
```

`markpublish` nummeriert die Unterkapitel automatisch konsistent als `2.1` und `2.2`.

4.2.3 Übergeordnete Abschnitte (Parts / Blöcke)

Für Hauptabschnitte oder Anhangsblöcke, die mehrere Kapitel umfassen, verwenden Sie das Schlüsselwort `part:`:

```
chapters:
- part: "Anhänge"
  summary: "Ergänzende Tabellen und Referenzen."
  divider_page: true          # Große Trennseite für den gesamten Anhang
  autonum: "none"             # Keine vorangestellte Ziffer
  chapters:
  - file: "chapters/appendix_a.md"
    title: "Anhang A: Referenz"
  - file: "chapters/appendix_b.md"
    title: "Anhang B: Glossar"
```

Der Part-Titel (z. B. *"Anhänge"*) wird automatisch in die laufenden 2-zeiligen Kopfzeilen der Einzelseiten übernommen.

KAPITEL 5

Das Template- und Design-System

3-stufige Auflösungshierarchie, CSS Paged Media und 2-zeilige Kopf-/Fußzeilen.

AUF EINEN BLICK

5.1 Theme-basierte Verzeichnisstruktur	17
5.2 Die 3-stufige Auflösungs-Hierarchie	17
5.3 Statische Texte und Sprache	18
5.4 2-zeilige Kopf- und Fußzeilen	22

5 Das Template- und Design-System

Das Template-System von `markpublish` ermöglicht vollständige visuelle Anpassbarkeit bei gleichzeitiger Wartbarkeit.

5.1 Theme-basierte Verzeichnisstruktur

Templates sind nach Theme gruppiert, darunter liegt das Ausgabeformat. So gehören alle Dateien eines Designs zusammen und ein Theme lässt sich als Ganzes kopieren, weitergeben oder versionieren:

```
templates/
├── default/                                # Theme-Name
│   ├── pdf/
│   │   ├── layout.html                    # HTML-Grundgerüst
│   │   ├── styles.css                    # CSS Paged Media & 2-zeilige Kopf-/Fußzeilen
│   │   ├── cover.html                    # Deckblatt-Template
│   │   ├── part_divider.html              # Trennseite für übergeordnete Parts
│   │   ├── chapter_divider.html          # Trennseite für Kapitel mit Mini-TOC
│   │   └── toc.html                      # Globales Inhaltsverzeichnis
│   └── html/
│       ├── layout.html                    # Standalone Web-Layout
│       ├── styles.css                    # Screen/Responsive Stylesheet
│       └── ...
```

Hinweis

Bis einschließlich Version 0.1 lag die Struktur umgekehrt (`templates/pdf/default`). Templates im alten Layout werden weiterhin gefunden, `markpublish templates` markiert sie in der Spalte *Layout* und gibt einen Hinweis aus. Verschieben Sie sie bei Gelegenheit — die alte Auflösung entfällt in einer künftigen Version.

5.2 Die 3-stufige Auflösungs-Hierarchie

Beim Suchen nach einem Template (z. B. `default/pdf`) gilt folgende strikte Priorität:

1. User-Verzeichnis (`~/markpublish/templates/default/pdf/`)
| (höchste Priorität)
▼
2. Gemeinsamer / Projekt-Ordner (`<templates_dir>/default/pdf/`)
|
▼
3. Paket Built-in (im `markpublish` Python-Paket)

5.2.1 Konfiguration des gemeinsamen Template-Ordners

Sie können den gemeinsamen Template-Pfad auf 4 Wegen festlegen: 1. **CLI-Parameter:** `markpublish build --templates-dir /pfad/zu/templates` 2. **In `markpublish.yaml`:** `templates_dir: "/.custom-templates"` 3.

Umgebungsvariable: `MARKPUBLISH_TEMPLATES_DIR=/pfad/zu/templates` 4. Automatischer Fallback: `./templates` im Projektordner.

5.3 Statische Texte und Sprache

Die festen Beschriftungen — Überschrift des Inhaltsverzeichnisses, Kapitel-Marken, Cover-Labels, Seitenzahl-Fußzeile, Callout-Titel — stehen nicht im Template, sondern in `i18n.yaml`-Dateien. Welche Sprache gilt, bestimmt `document.language`:

```
document:
  title: "User Guide"
  language: "en"      # steuert Beschriftungen, Callout-Titel und Datumsformat
```

Mitgeliefert sind `de` und `en`. Regionale Formen werden zugeordnet (`de-AT` → `de`), eine unbekannte Sprache fällt auf Englisch zurück.

Hinweis

`i18n` bezeichnet die Quellen — die Dateien und den YAML-Eintrag, jeweils über alle Sprachen. `labels` ist das daraus aufgelöste Ergebnis für *ein* Dokument in *einer* Sprache: das, was im Template unter `{{ labels.chapter }}` ankommt.

5.3.1 Die 4-stufige i18n-Kaskade

Alle vier Ebenen sind **identisch aufgebaut**: Sprachcode auf oberster Ebene, darunter die Texte. Jede tiefere Ebene überschreibt die höher liegende — und zwar **nur die Schlüssel, die sie tatsächlich setzt**. Alles andere bleibt, wie es weiter oben steht:

1. Programm	<code>markpublish/i18n.yaml</code> vollständig, <code>de</code> + <code>en</code> ▼
2. Theme	<code><templates>/<theme>/i18n.yaml</code> gilt für alle Zielformate des Themes ▼
3. Zielformat	<code><templates>/<theme>/<target>/i18n.yaml</code> nur für PDF bzw. nur für HTML ▼
4. Dokument	<code>document.i18n</code> in der <code>markpublish.yaml</code> höchste Priorität

Ebene 1 ist die einzige, die vollständig sein muss. Sie legt zusätzlich Englisch unter die Dokumentsprache, damit jeder Schlüssel garantiert auflöst. Die Ebenen 2–4 sind reine Overrides.

! Wichtig

Die Ebenen 2–4 greifen **nur für die gewählte Sprache**. Ein Theme, das einen `en:`-Block definiert, verändert eine deutsche Ausgabe nicht — sonst würden englische Theme-Texte in fremdsprachige Dokumente durchschlagen.

5.3.2 Das gemeinsame Format

```
# templates/mytheme/i18n.yaml
de:
  part: "Abschnitt"
  chapter_toc_title: "Auf dieser Seite"
en:
  part: "Section"
  chapter_toc_title: "On this page"
```

Der Sonderschlüssel `""` gilt für **jede** Sprache und wird vor dem sprachspezifischen Block angewendet — für Begriffe, die unabhängig von der Dokumentsprache gleich heißen:

```
"":
  version: "Rev."      # in jeder Sprache "Rev."
de:
  part: "Abschnitt"
```

Wer nur eine Sprache pflegt, darf die Spracheebene auch weglassen; die flache Form ist gleichbedeutend mit

```
"":
```

```
part: "Abschnitt"      # entspricht: "":\n part: "Abschnitt"
```

Regionale Blöcke schlagen den Basis-Block: bei `language: "de-AT"` gewinnt `de-at:` über `de:`. Fehlt eine `i18n.yaml`, ist das kein Fehler — ein Theme ohne eigene Texte ist der Normalfall. Ist sie vorhanden, aber fehlerhaft, bricht der Build mit Angabe der Datei ab, statt still die Standardtexte zu verwenden.

5.3.3 Beispiel: PDF und HTML unterschiedlich beschriften

```
templates/mytheme/
├─ i18n.yaml          de: chapter: "Kapitel"
├─ pdf/
│   └─ i18n.yaml      de: chapter: "Kap."      ← nur im PDF
└─ html/
    └─ i18n.yaml      (leer → erbt "Kapitel")
```

Das mitgelieferte Theme `default` bringt alle drei Dateien als **auskommentiertes Muster** mit: `templates/default/i18n.yaml`, `default/pdf/i18n.yaml` und `default/html/i18n.yaml`. Sie sind bewusst wirkungslos — das Standard-Theme soll exakt wie der Programmstandard aussprechen. Kommentieren Sie aus, was Sie ändern möchten.

5.3.4 Ebene 4: Texte für ein einzelnes Dokument

In der `markpublish.yaml` unter `document.i18n`, mit demselben Aufbau:

```
document:
  language: "de"
  i18n:
    de:
      part: "Abschnitt"
      chapter_toc_title: "Auf einen Blick"
```

Genau so ist dieses Handbuch konfiguriert — die Trennseiten zeigen deshalb *Auf einen Blick* statt *Inhalt dieses Kapitels* und der Anhang-Block *Abschnitt* statt *Teil*.

Dieselbe Stelle dient dazu, eine noch nicht mitgelieferte Sprache vollständig selbst zu setzen.

5.3.5 Die aufgelöste Tabelle ansehen

Bei vier Ebenen ist nicht immer offensichtlich, woher ein Text kommt. `markpublish labels` zeigt das Ergebnis samt Herkunft:

<code>markpublish labels</code>	<code># alle Schlüssel, Ziel PDF</code>
<code>markpublish labels --target html</code>	<code># Kaskade für die HTML-Ausgabe</code>
<code>markpublish labels --overridden</code>	<code># nur das, was von Ebene 2-4 kommt</code>

5.3.6 Verfügbare Schlüssel

SCHLÜSSEL	DE	EN
<code>toc_title</code>	Inhaltsverzeichnis	Table of Contents
<code>toc_sidebar</code>	Inhalt	Contents
<code>chapter_toc_title</code>	Inhalt dieses Kapitels	In this chapter
<code>chapter</code>	Kapitel	Chapter
<code>part</code>	Teil	Part
<code>author</code>	Autor	Author
<code>status</code>	Status	Status
<code>version</code>	Version	Version
<code>date</code>	Datum	Date
<code>copyright</code>	Copyright	Copyright
<code>page</code>	Seite	Page
<code>page_of</code>	von	of
<code>alert_note</code>	Hinweis	Note
<code>alert_tip</code>	Tipp	Tip
<code>alert_important</code>	Wichtig	Important
<code>alert_warning</code>	Warnung	Warning
<code>alert_caution</code>	Achtung	Caution

Eine neue Sprache legen Sie an, indem Sie in `markpublish/i18n.yaml` einen vollständigen Block ergänzen — oder, ohne das Paket anzufassen, auf Ebene 2–4 alle Schlüssel unter dem gewünschten Sprachcode setzen.

💡 Tipp

In eigenen Templates greifen Sie mit `{{ labels.chapter }}` auf das Ergebnis zu — auch in `styles.css`, das durch dieselbe Jinja-Umgebung läuft. So ist die Fußzeile `"{{ labels.page }} " counter(page) "`
`{{ labels.page_of }}` `" counter(pages) gebaut.`

5.4 2-zeilige Kopf- und Fußzeilen

Kopf- und Fußzeilen werden im Template via CSS Paged Media `@page` Margin-Boxes definiert:

```
@page {
  size: A4;
  margin: 25mm 20mm 25mm 20mm;

  @top-left {
    content: "{{ document.title }}" string(current_section);
    font-size: 8pt;
    white-space: pre-wrap;
    border-bottom: 0.5pt solid #cbd5e1;
  }

  @top-right {
    content: "v{{ document.version }}" A{{ document.date }}";
    font-size: 8pt;
    text-align: right;
    white-space: pre-wrap;
    border-bottom: 0.5pt solid #cbd5e1;
  }

  @bottom-left {
    content: "{{ document.author or '' }}" A{{ document.copyright or document.status or '' }}";
    font-size: 8pt;
    white-space: pre-wrap;
    border-top: 0.5pt solid #cbd5e1;
  }

  @bottom-right {
    content: "Seite " counter(page) " von " counter(pages);
    font-size: 8pt;
    text-align: right;
    border-top: 0.5pt solid #cbd5e1;
  }
}
```

KAPITEL 6

Markdown-Features & Syntax

Tabellen, Admonitions/Callouts, Pygments Syntax-Highlighting und Tasklisten.

6 Markdown-Features & Syntax

markpublish beinhaltet eine leistungsfähige Markdown-Engine mit voller Unterstützung moderner Dokumentations-Elemente.

6.1 Code-Blöcke & Syntax Highlighting

Quellcode wird durch Pygments hervorgehoben:

```
from markpublish.config.loader import load_config
from markpublish.markdown.engine import MarkdownPipeline

# Manifest Laden
config = load_config("markpublish.yaml")
print(f"Building: {config.document.title}")
```

6.2 Tabellen

Tabellen werden mit sauberem CSS-Zebrawuster und Seitenumbruchschatz gerendert:

PARAMETER	TYP	STANDARD	BESCHREIBUNG
title	String	<i>Pflicht</i>	Haupttitel des Dokuments
author	String	None	Name des Autors
status	String	None	Dokumentstatus (z.B. "Freigegeben", "Draft")
copyright	String	None	Copyright-Vermerk (z.B. "© 2026 Frank Winter")
version	String	"1.0.0"	Versionskennung

6.3 GitHub-Style Callout-Boxen (Alerts)

markpublish unterstützt die gängige **GitHub-Alert-Syntax** mit fünf semantischen Typen. Die Icons stammen dabei direkt aus dem Template:

Hinweis

Dies ist eine informative Notiz mit blauem Akzent und passendem Info-Icon.

Tipp

Nutzen Sie `markpublish build --target all`, um PDF und HTML parallel in einem Durchgang zu erstellen.

Wichtig

GitHub-Callouts werden automatisch anhand der Dokumentensprache lokalisiert (z. B. *Hinweis*, *Tipp*, *Wichtig*, *Warnung*, *Achtung*).

Warnung

Achten Sie bei relativen Bildpfaden darauf, dass diese relativ zur jeweiligen Markdown-Datei liegen.

Achtung

Fehlerhafte YAML-Einrückungen führen zu Abbruchfehlern beim Parsen des Manifests.

Eigener Titel

Sie können nach dem Alert-Typ auch einen individuellen Titel angeben, der die Standardbeschriftung überschreibt.

Alternativ wird auch weiterhin die klassische `!!! note` -Syntax unterstützt.

6.4 Tasklisten

- ☒ Projekt initialisieren
- ☒ Kapitel schreiben
- ☐ Dokument veröffentlichen

ABSCHNITT

Anhänge

Vollständige Spezifikations-Tabellen, Troubleshooting und Best Practices.

6.5 Anhang A: YAML-Schema-Referenz

Vollständige Übersicht aller Konfigurationsoptionen in `markpublish.yaml`.

6.5.1 Dokumenten-Eigenschaften (`document`)

SCHLÜSSEL	TYP	STANDARD	BESCHREIBUNG
<code>title</code>	String	<i>Pflicht</i>	Titel der Publikation
<code>subtitle</code>	String	<code>null</code>	Untertitel
<code>summary</code>	String	<code>null</code>	Zusammenfassung für Deckblatt
<code>author</code>	String	<code>null</code>	Name des Autors
<code>status</code>	String	<code>null</code>	Dokumentstatus (z. B. "Entwurf", "Freigegeben")
<code>copyright</code>	String	<code>null</code>	Copyright-Angabe (z. B. "© 2026 Frank Winter")
<code>date</code>	String	<code>"auto"</code>	Datum oder <code>"auto"</code> für Tagesdatum
<code>version</code>	String	<code>"1.0.0"</code>	Versionskennung
<code>language</code>	String	<code>"de"</code>	ISO-Sprachcode (<code>de</code> , <code>en</code> , ...). Steuert Template-Beschriftungen, Callout-Titel und Datumsformat
<code>i18n</code>	Map	<code>{}</code>	Ebene 4 der i18n-Kaskade. Gleicher Aufbau wie die <code>i18n.yaml</code> : Sprachcode, darunter die Texte. Siehe Kapitel 5
<code>cover</code>	Bool	<code>true</code>	Deckblatt aktivieren/deaktivieren
<code>toc</code>	Bool	<code>true</code>	Globales Inhaltsverzeichnis aktivieren
<code>autonum_type</code>	String	<code>"decimal"</code>	<code>"decimal"</code> , <code>"roman"</code> , <code>"legal"</code> , <code>"none"</code>
<code>header</code>	Bool	<code>true</code>	2-zeilige Kopfzeile aktivieren
<code>footer</code>	Bool	<code>true</code>	2-zeilige Fußzeile aktivieren

6.5.2 Kapitel- und Part-Eigenschaften (`chapters`)

SCHLÜSSEL	TYP	STANDARD	BESCHREIBUNG
<code>file</code>	String	<code>null</code>	Pfad zur Markdown-Datei
<code>title</code>	String	<code>null</code>	Überschreibt den Titel der Datei
<code>summary</code>	String	<code>null</code>	Zusammenfassung für Trennseite
<code>part</code>	String	<code>null</code>	Deklariert einen übergeordneten Part
<code>divider_page</code>	Bool	<code>false</code>	Fügt eine separate Trennseite ein
<code>toc</code>	Bool/Int	<code>false</code>	Lokales Kapitel-TOC (z. B. <code>2</code> für max. Tiefe)
<code>autonom</code>	String	<code>null</code>	Lokaler Override des Nummerierungsstils
<code>chapters</code>	Liste	<code>[]</code>	Verschachtelte Unterkapitel

6.6 Anhang B: Fehlerbehebung & FAQ

Häufige Fragen und Lösungen bei der Dokumentenerstellung.

6.6.1 WeasyPrint & GTK-Laufzeitumgebung auf Windows

6.6.1.1 Problem: `cannot load library 'libgobject-2.0-0'`

WeasyPrint benötigt auf Windows-Systemen die nativen C-Bibliotheken von Pango und GTK.

Lösung: `markpublish` sucht automatisch nach installierten GTK-Umgebungen (wie MSYS2, GTK3-Runtime, darktable oder Inkscape). Sollten diese fehlen, installieren Sie das offizielle GTK3-Runtime-Paket: - Download: [GTK-for-Windows-Runtime-Environment-Installer](#) - Alternativ via MSYS2: `pacman -S mingw-w64-x86_64-pango`

6.6.2 Bilder werden im PDF nicht angezeigt

6.6.2.1 Problem: Leere Bildrahmen oder fehlende Grafiken

WeasyPrint benötigt gültige relative Pfade oder absolute File-URIs.

Lösung: Geben Sie relative Bildpfade immer relativ zur jeweiligen Markdown-Datei an:

```
![Architekturdiagramm](images/architecture.png)
```

`markpublish` löst relative Pfade automatisch zu absoluten Datei-URLs auf.

6.6.3 Inhaltsverzeichnis zeigt falsche Seitenzahlen

Das PDF-Rendering erfolgt in mehreren Layout-Durchläufen (Zweipass-Rendering von WeasyPrint). Stellen Sie sicher, dass alle internen Überschriften-IDs eindeutig sind – `markpublish` übernimmt diese Eindeutigkeit automatisch über seinen internen Slug-Generator.